5.3 Oracle 시스템 고급 기능 분석

개요

Oracle 팰릿은 Composable Finance 생태계의 핵심 가격 피드 인프라로, 외부 데이터의 온체인 통합을 위한 정교한 메커니즘을 구현합니다. 이 문서는 다중 데이터 소스 집계, 이상치 제거 알고리즘, MEV 방지 전략, 인센티브 구조 등의 고급 기능을 심층 분석합니다.

5.3.1 다중 데이터 소스 집계 및 이상치 제거 알고리즘 분석

가격 집계 시스템 아키텍처

Oracle 시스템은 다단계 집계 프로세스를 통해 신뢰할 수 있는 가격 데이터를 생성합니다.

가격 제출 및 임시 저장

```
#[pallet::storage]

#[pallet::getter(fn pre_prices)]

pub  type PrePrices<T: Config> = StorageMap<

_,

Blake2_128Concat,

T::AssetId,

BoundedVec<PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>, T::MaxPrePrice

ValueQuery,

>;
```

```
#[derive(Encode, Decode, MaxEncodedLen, Clone, Copy, Default, Debug, PartialEq,
pub struct PrePrice<PriceValue, BlockNumber, AccountId> {
pub price: PriceValue, // 제출된 가격
pub block: BlockNumber, // 제출 블록
pub who: AccountId, // 제출자
}
```

가격 제출 프로세스

```
#[pallet::call_index(6)]
#[pallet::weight((T::WeightInfo::submit_price(T::MaxAnswerBound::get()), Dispatc
pub fn submit_price(
origin: OriginFor<T>,
price: T::PriceValue,
asset_id: T::AssetId,
) -> DispatchResultWithPostInfo {
let who = ensure_signed(origin)?;
let signer = Self::signer_to_controller(&who).unwrap_or(who.clone());
// 스테이킹 잔액 확인
let author_stake = Self::oracle_stake(&signer).unwrap_or_default();
ensure!(author_stake >= T::MinStake::get(), Error::<T>::NotEnoughStake);
// 자산 정보 확인
```

```
let asset_info = Self::asset_info(asset_id).ok_or(Error::<T>::UnknownAssetId)?;
// 중복 제출 방지
let mut current_prices = Self::pre_prices(asset_id);
if current_prices.iter().any(|candidate| candidate.who == signer) {
return Err(Error::<T>::AlreadySubmitted.into());
}
// 최대 응답 수 제한
if current_prices.len() as u32 >= asset_info.max_answers {
return Err(Error::<T>::MaxPrices.into());
}
// 슬래시 예약 (부정확한 답변에 대한 보증금)
AnswerInTransit::<T>::mutate(&signer, |transit| {
*transit = Some(transit.unwrap_or_default().saturating_add(asset_info.slash));
});
// 가격 데이터 저장
let pre_price = PrePrice {
price,
block: frame_system::Pallet::<T>::block_number(),
who: signer.clone(),
};
current_prices.try_push(pre_price).map_err(|_| Error::<T>::MaxPrices)?;
PrePrices::<T>::insert(asset_id, current_prices);
Self::deposit_event(Event::PriceSubmitted(signer, asset_id, price));
```

```
Ok(Pays::No.into()) // 성공 시 가스 수수료 면제
}
```

중앙값 기반 집계 알고리즘

기본 중앙값 계산

```
pub fn get_median_price(
prices: &[PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>],
) -> Option<T::PriceValue> {
if prices.is_empty() {
return None;
}
// 가격 추출 및 정렬
let mut price_values: Vec<T::PriceValue> = prices.iter().map(|p| p.price).coll
price_values.sort();
let len = price_values.len();
// 중앙값 계산
if len % 2 == 0 {
// 짝수 개의 경우 중간 두 값의 평균
let mid1 = price_values[len / 2 - 1];
let mid2 = price_values[len / 2];
Some((mid1 + mid2) / 2u32.into())
} else {
```

```
// 홀수 개의 경우 중간값
Some(price_values[len / 2])
}
}
```

고급 이상치 제거 알고리즘

임계값 기반 필터링

```
pub fn get_price_after_filtering_by_threshold(
prices: &[PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>],
asset_info: &AssetInfo<Percent, T::BlockNumber, BalanceOf<T>>,
) -> Option<T::PriceValue> {
// 1단계: 중앙값 계산
let median_price = Self::get_median_price(prices)?;
let mut sum_of_price = T::PriceValue::zero();
let mut number_of_prices = 0u32;
// 2단계: 임계값 내의 가격만 필터링
for answer in prices {
// 중앙값 대비 정확도 계산
let accuracy: Percent = if answer.price < median_price {
PerThing::from_rational(answer.price, median_price)
} else {
// 중앙값보다 높은 경우 조정된 정확도 계산
```

```
let adjusted_number = median_price.saturating_sub(answer.price - median_price);
PerThing::from_rational(adjusted_number, median_price)
};
let min_accuracy = asset_info.threshold;
// 임계값 이상의 정확도를 가진 가격만 포함
if accuracy >= min_accuracy {
sum_of_price += answer.price;
number_of_prices += 1;
}
}
// 3단계: 필터링된 가격들의 평균 계산
if number_of_prices > 0 {
Some(sum_of_price / number_of_prices.into())
} else {
// 모든 가격이 필터링된 경우 중앙값 반환
Some(median_price)
}
}
```

가중치 기반 집계

```
pub fn get_weighted_price(
prices: &[PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>],
```

```
weights: &[u128],
) -> Option<T::PriceValue> {
if prices.len() != weights.len() || prices.is_empty() {
return None;
}
let weights_sum: u128 = weights.iter().sum();
if weights_sum.is_zero() {
// 가중치가 모두 0인 경우 단순 평균
let sum: u128 = prices.iter().map(|p| p.price.into()).sum();
return Some((sum / prices.len() as u128).into());
}
// 가중 평균 계산
let weighted_sum: u128 = prices
.iter()
.map(|e| -> u128 { e.price.into() })
.zip(weights)
.filter_map(|(price, weight)| {
if *weight > 0 {
Some(price * weight)
} else {
None
}
})
```

```
.sum();
Some((weighted_sum / weights_sum).into())
}
```

자동 가격 집계 및 업데이트

```
fn update_prices_and_rewards(block: T::BlockNumber) -> Weight {
let mut total_weight = Weight::zero();
// 모든 자산에 대해 가격 업데이트 확인
for (asset_id, asset_info) in AssetsInfo::<T>::iter() {
// 블록 간격 확인
if block % asset_info.block_interval == T::BlockNumber::zero() {
let pre_prices = PrePrices::<T>::get(asset_id);
// 최소 응답 수 확인
if pre_prices.len() >= asset_info.min_answers as usize {
// 가격 집계 실행
if let Some(aggregated_price) = Self::get_price_after_filtering_by_threshold(
&pre_prices,
&asset_info,
) {
// 최종 가격 업데이트
Self::update_price(asset_id, aggregated_price, block);
// 리워드 및 슬래시 처리
```

```
Self::handle_payout(&pre_prices, aggregated_price, asset_id, &asset_info);

// 가격 이력 업데이트 (TWAP용)

Self::update_price_history(asset_id, Price {

price: aggregated_price,

block,

});

}

// 임시 가격 데이터 정리

PrePrices::<T>::remove(asset_id);

total_weight = total_weight.saturating_add(T::WeightInfo::aggregate_prices());

}

}

}

total_weight

}
```

5.3.2 오라클 보안 메커니즘 및 MEV 방지 전략 분석

스테이킹 기반 보안 모델

최소 스테이킹 요구사항

```
#[pallet::call_index(1)]

#[pallet::weight(T::WeightInfo::set_signer())]
```

```
pub fn set_signer(
origin: OriginFor<T>,
who: T::AccountId,
signer: T::AccountId,
) -> DispatchResultWithPostInfo {
let controller = ensure_signed(origin)?;
// 컨트롤러와 서명자가 다른지 확인
ensure!(controller != signer, Error::<T>::ControllerUsedAsSigner);
// 기존 매핑 확인
ensure!(
!SignerToController::<T>::contains_key(&signer),
Error::<T>::SignerUsed
);
ensure!(
!ControllerToSigner::<T>::contains_key(&controller),
Error::<T>::ControllerUsed
);
// 최소 스테이킹 금액 예치
let min_stake = T::MinStake::get();
T::Currency::reserve(&controller, min_stake)?;
// 매핑 관계 설정
SignerToController::<T>::insert(&signer, &controller);
ControllerToSigner::<T>::insert(&controller, &signer);
```

```
OracleStake::<T>::insert(&controller, min_stake);

Self::deposit_event(Event::SignerSet(controller.clone(), signer.clone()));

Self::deposit_event(Event::StakeAdded(controller, min_stake));

Ok(().into())

}
```

슬래시 예약 시스템

```
// 가격 제출 시 슬래시 금액 예약

AnswerInTransit::<T>::mutate(&signer, |transit| {

*transit = Some(transit.unwrap_or_default().saturating_add(asset_info.slash));

});


// 부정확한 답변에 대한 슬래시 실행

fn slash_oracle(

who: &T::AccountId,

slash_amount: BalanceOf<T>,

asset_id: T::AssetId,

) -> DispatchResult {

// 현재 스테이킹에서 슬래시 금액 차감

let current_stake = Self::oracle_stake(who).unwrap_or_default();

let new_stake = current_stake.saturating_sub(slash_amount);

if new_stake.is_zero() {

// 스테이킹이 모두 소진된 경우 오라클 자격 박탈
```

```
OracleStake::<T>::remove(who);

// 매핑 관계 제거

if  let Some(signer) = Self::controller_to_signer(who) {

SignerToController::<T>::remove(&signer);

ControllerToSigner::<T>::remove(who);

}

} else {

OracleStake::<T>::insert(who, new_stake);

}

// 슬래시된 자산을 국고로 전송

T::Currency::repatriate_reserved(

who,

&T::TreasuryAccount::get(),

slash_amount,

BalanceStatus::Free,

)?;

// 예약된 슬래시 금액 해제

AnswerInTransit::<T>::mutate(who, |transit| {

if  let Some(current) = transit {

*transit = Some(current.saturating_sub(slash_amount));

if  transit.unwrap_or_default().is_zero() {

*transit = None;

}
```

```
}
});
Self::deposit_event(Event::UserSlashed(who.clone(), asset_id, slash_amount));
Ok(())
}
```

MEV 방지 메커니즘

지연된 스테이킹 해제

```
#[pallet::call_index(4)]
#[pallet::weight(T::WeightInfo::remove_stake())]
pub fn remove_stake(origin: OriginFor<T>) -> DispatchResultWithPostInfo {
let who = ensure_signed(origin)?;
let stake = Self::oracle_stake(&who).ok_or(Error::<T>::NotAnOracle)?;
// 즉시 스테이킹 상태에서 제거 (새로운 가격 제출 불가)
OracleStake::<T>::remove(&who);
// 지연된 해제를 위한 블록 번호 계산
let unlock_block = frame_system::Pallet::<T>::block_number()
.saturating_add(T::StakeLock::get());
// 해제 요청 저장
DeclaredWithdraws::<T>::insert(&who, unlock_block);
Self::deposit_event(Event::StakeRemoved(who, stake));
Ok(().into())
```

```
}

#[pallet::call_index(5)]
#[pallet::weight(T::WeightInfo::reclaim_stake())]
pub fn reclaim_stake(origin: OriginFor<T>) -> DispatchResultWithPostInfo {
let who = ensure_signed(origin)?;
let unlock_block = Self::declared_withdraws(&who).ok_or(Error::<T>::NoStakeToWi
let current_block = frame_system::Pallet::<T>::block_number();
// 해제 가능 시점 확인
ensure!(current_block > unlock_block, Error::<T>::StakeLocked);
// 예약된 자산 해제
let amount = T::Currency::unreserve(&who, BalanceOf::<T>::max_value());
// 해제 요청 제거
DeclaredWithdraws::<T>::remove(&who);
// 매핑 관계 정리
if let Some(signer) = Self::controller_to_signer(&who) {
SignerToController::<T>::remove(&signer);
ControllerToSigner::<T>::remove(&who);
}
Self::deposit_event(Event::StakeReclaimed(who, amount));
Ok(().into())
}
```

중복 제출 방지

```
// 동일 오라클의 중복 제출 방지
let mut current_prices = Self::pre_prices(asset_id);
if current_prices.iter().any(|candidate| candidate.who == signer) {
return Err(Error::<T>::AlreadySubmitted.into());
}
```

Operational 클래스 트랜잭션

```
// 가격 제출을 Operational 클래스로 분류하여 우선 처리
#[pallet::call_index(6)]
#[pallet::weight((T::WeightInfo::submit_price(T::MaxAnswerBound::get()), Dispatc
pub fn submit_price(
origin: OriginFor<T>,
price: T::PriceValue,
asset_id: T::AssetId,
) -> DispatchResultWithPostInfo {
// ... 가격 제출 로직
Ok(Pays::No.into()) // 성공 시 가스 수수료 면제
}
```

가격 조작 방지 테스트

```rust
#[test]
fn halborn_test_price_manipulation() {
new_test_ext().execute_with(|| {
const ASSET_ID: u128 = 0;
// 정상적인 가격 범위 설정
let normal_prices = vec![100, 101, 102, 99, 98];
let manipulated_price = 1000; // 10배 높은 가격
// 여러 오라클이 정상 가격 제출
for (i, price) in normal_prices.iter().enumerate() {
let oracle_account = AccountId::from([i as u8; 32]);
setup_oracle(&oracle_account);
assert_ok!(Oracle::submit_price(
RuntimeOrigin::signed(oracle_account),
*price,
ASSET_ID
));
}
// 악의적 오라클이 조작된 가격 제출
let malicious_oracle = AccountId::from([99u8; 32]);
setup_oracle(&malicious_oracle);
assert_ok!(Oracle::submit_price(
RuntimeOrigin::signed(malicious_oracle),
```

```
manipulated_price,
ASSET_ID
));
// 가격 집계 실행
Oracle::on_initialize(10);
// 중앙값 기반 집계로 조작 시도 무효화 확인
let final_price = Oracle::prices(ASSET_ID).price;
assert!(final_price < 150); // 조작된 가격이 반영되지 않음
// 악의적 오라클 슬래시 확인
assert!(Oracle::oracle_stake(&malicious_oracle).unwrap_or_default() < T::MinStak
});
}
```

5.3.3 가격 피드 인센티브 구조 및 리워드 분배 시스템 분석

리워드 추적 시스템

RewardTracker 구조체

```
pub struct RewardTracker<Balance, Moment> {
pub period: Moment, // 리워드 기간 (보통 1년)
pub start: Moment, // 시작 시점
pub total_already_rewarded: Balance, // 이미 지급된 총 리워드
pub current_block_reward: Balance, // 현재 블록당 리워드
```

```
pub total_reward_weight: Balance, // 총 리워드 가중치
}

#[pallet::storage]
#[pallet::getter(fn reward_tracker_store)]
pub type RewardTrackerStore<T: Config> = StorageValue<_, RewardTracker<BalanceO
```

리워드 시스템 초기화 및 조정

```
#[pallet::call_index(2)]
#[pallet::weight(T::WeightInfo::adjust_rewards())]
pub fn adjust_rewards(
origin: OriginFor<T>,
annual_cost_per_oracle: BalanceOf<T>,
num_ideal_oracles: u8,
) -> DispatchResultWithPostInfo {
T::RewardOrigin::ensure_origin(origin)?;
let current_time = T::Time::now();
// 연간 총 리워드 계산
let new_annual_reward = annual_cost_per_oracle
.saturating_mul(num_ideal_oracles.into());
let (total_already_rewarded, period_left) = if let Some(reward_tracker) = Self:
// 기존 리워드 추적기가 있는 경우
```

```
let elapsed = current_time.saturating_sub(reward_tracker.start);
let period_left = reward_tracker.period.saturating_sub(elapsed);
(reward_tracker.total_already_rewarded, period_left)
} else {
// 새로운 리워드 추적기 생성
(BalanceOf::<T>::zero(), T::Time::get_period())
};
// 남은 기간 동안의 블록 수 계산
let remaining_blocks_in_year = period_left
.checked_div(&T::MsPerBlock::get())
.ok_or(Error::<T>::MathError)?;
// 블록당 리워드 계산
let current_block_reward = new_annual_reward
.saturating_sub(total_already_rewarded)
.checked_div(&remaining_blocks_in_year.into())
.ok_or(Error::<T>::MathError)?;
// 총 리워드 가중치 계산
let total_reward_weight: BalanceOf<T> = AssetsInfo::<T>::iter()
.map(|(_, asset_info)| asset_info.reward_weight)
.fold(BalanceOf::<T>::zero(), |acc, weight| acc.saturating_add(weight));
// 리워드 추적기 업데이트
let new_reward_tracker = RewardTracker {
period: T::Time::get_period(),
```

```
start: current_time,
total_already_rewarded,
current_block_reward,
total_reward_weight,
};
RewardTrackerStore::<T>::put(new_reward_tracker);
Self::deposit_event(Event::RewardingAdjustment(
annual_cost_per_oracle,
num_ideal_oracles,
));
Ok(().into())
}
```

정확도 기반 리워드 분배

리워드 지급 로직

```
fn handle_payout(
pre_prices: &[PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>],
aggregated_price: T::PriceValue,
asset_id: T::AssetId,
asset_info: &AssetInfo<Percent, T::BlockNumber, BalanceOf<T>>,
) -> DispatchResult {
for answer in pre_prices {
```

```
// 정확도 계산
let accuracy: Percent = if answer.price < aggregated_price {
PerThing::from_rational(answer.price, aggregated_price)
} else {
let adjusted_number = aggregated_price
.saturating_sub(answer.price - aggregated_price);
PerThing::from_rational(adjusted_number, aggregated_price)
};
let min_accuracy = asset_info.threshold;
if accuracy < min_accuracy {
// 부정확한 답변에 대한 슬래시
Self::slash_oracle(&answer.who, asset_info.slash, asset_id)?;
} else {
// 정확한 답변에 대한 리워드
Self::reward_oracle(&answer.who, asset_id, asset_info)?;
}
}
Ok(())
}
```

가중치 기반 리워드 계산

```
fn reward_oracle(
```

```
who: &T::AccountId,
asset_id: T::AssetId,
asset_info: &AssetInfo<Percent, T::BlockNumber, BalanceOf<T>>,
) -> DispatchResult {
if  let Some(reward_tracker) = Self::reward_tracker_store() {
// 자산별 리워드 가중치
let  asset_reward_weight = asset_info.reward_weight;
// 총 가중치 대비 비율 계산
let  reward_ratio = Percent::from_rational(
asset_reward_weight,
reward_tracker.total_reward_weight,
);
// 블록당 리워드에서 해당 자산의 비율만큼 계산
let  reward_amount = reward_ratio
.mul_floor(reward_tracker.current_block_reward);
if !reward_amount.is_zero() {
// 리워드 지급
T::Currency::deposit_creating(who, reward_amount)?;
// 누적 리워드 업데이트
AccumulatedRewardsPerAsset::<T>::mutate(asset_id, |accumulated| {
*accumulated = Some(
accumulated.unwrap_or_default().saturating_add(reward_amount)
);
```

```
});
// 총 지급 리워드 업데이트
RewardTrackerStore::<T>::mutate(|tracker| {
if let Some(ref mut tracker) = tracker {
tracker.total_already_rewarded =
tracker.total_already_rewarded.saturating_add(reward_amount);
}
});
Self::deposit_event(Event::OracleRewarded(
who.clone(),
asset_id,
reward_amount,
));
}
}
Ok(())
}
```

동적 리워드 조정

자산별 누적 리워드 추적

```
#[pallet::storage]
#[pallet::getter(fn accumulated_rewards_per_asset)]
```

```
pub type AccumulatedRewardsPerAsset<T: Config> =
StorageMap<_, Blake2_128Concat, T::AssetId, BalanceOf<T>>;


// 자산별 리워드 분배 현황 조회
pub fn get_asset_reward_distribution(asset_id: T::AssetId) -> Option<BalanceOf
Self::accumulated_rewards_per_asset(asset_id)
}
```

리워드 풀 관리

```
impl<T: Config> Pallet<T> {
/// 오라클 팰릿 전용 계정 ID 생성
pub fn account_id() -> T::AccountId {
T::PalletId::get().into_account_truncating()
}
/// 리워드 풀 잔액 확인
pub fn reward_pool_balance() -> BalanceOf<T> {
T::Currency::free_balance(&Self::account_id())
}
/// 리워드 지급 가능 여부 확인
pub fn can_pay_rewards(amount: BalanceOf<T>) -> bool {
Self::reward_pool_balance() >= amount
}
```

```
}
```

인센티브 최적화 메커니즘

참여 오라클 수에 따른 동적 조정

```
fn calculate_optimal_reward_per_oracle() -> BalanceOf<T> {
let active_oracles = OracleStake::<T>::iter().count() as u32;
let ideal_oracles = T::IdealOracleCount::get();
if active_oracles < ideal_oracles {
// 오라클 수가 부족한 경우 리워드 증가
let shortage_ratio = Percent::from_rational(
ideal_oracles - active_oracles,
ideal_oracles,
);
// 기본 리워드에 부족분만큼 보너스 추가
let base_reward = T::BaseRewardPerOracle::get();
let bonus = shortage_ratio.mul_floor(base_reward);
base_reward.saturating_add(bonus)
} else {
// 충분한 오라클이 있는 경우 기본 리워드
T::BaseRewardPerOracle::get()
}
}
```

정확도 기반 리워드 스케일링

```
fn calculate_accuracy_bonus(
accuracy: Percent,
base_reward: BalanceOf<T>,
) -> BalanceOf<T> {
if accuracy >= Percent::from_percent(95) {
// 95% 이상 정확도: 50% 보너스
base_reward.saturating_add(base_reward / 2u32.into())
} else if accuracy >= Percent::from_percent(90) {
// 90% 이상 정확도: 25% 보너스
base_reward.saturating_add(base_reward / 4u32.into())
} else {
// 기본 리워드
base_reward
}
}
```

5.3.4 고급 보안 및 안정성 메커니즘

가격 조작 탐지

```
fn detect_price_manipulation(
prices: &[PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>],
```

```
) -> bool {
if prices.len() < 3 {
return false;
}
let mut price_values: Vec<T::PriceValue> = prices.iter().map(|p| p.price).coll
price_values.sort();
let median = price_values[price_values.len() / 2];
let q1 = price_values[price_values.len() / 4];
let q3 = price_values[3 * price_values.len() / 4];
let iqr = q3.saturating_sub(q1);
let outlier_threshold = iqr.saturating_mul(3u32.into()) / 2u32.into();
// 이상치 비율 계산
let outlier_count = price_values.iter().filter(|&&price| {
price < q1.saturating_sub(outlier_threshold) ||
price > q3.saturating_add(outlier_threshold)
}).count();
// 이상치가 전체의 30% 이상인 경우 조작 의심
outlier_count * 10 > prices.len() * 3
}
```

자동 비상 정지 메커니즘

```
fn emergency_pause_if_needed(asset_id: T::AssetId) -> DispatchResult {
```

```
let recent_prices = Self::price_history(asset_id);
if recent_prices.len() >= 5 {
let latest_prices: Vec<T::PriceValue> = recent_prices
.iter()
.rev()
.take(5)
.map(|p| p.price)
.collect();
// 급격한 가격 변동 감지
let max_price = *latest_prices.iter().max().unwrap();
let min_price = *latest_prices.iter().min().unwrap();
let volatility_ratio = Percent::from_rational(
max_price.saturating_sub(min_price),
min_price,
);
// 50% 이상 변동 시 비상 정지
if volatility_ratio > Percent::from_percent(50) {
EmergencyPause::<T>::insert(asset_id, true);
Self::deposit_event(Event::EmergencyPause(asset_id));
}
}
Ok(())
```

```
}
```

이러한 고급 기능들을 통해 Oracle 시스템은 신뢰할 수 있고 조작 저항성이 있는 가격 피드를 제공하며, 탈중앙화된 오라클 네트워크의 건전한 운영을 보장합니다.